**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**
**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА**

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

# ЗВІТ

## З ЛАБОРАТОРНОЇ РОБОТИ №2

з дисципліни «Функціональне програмування»

на тему: «Функції вищого порядку, lambda, композиція»

**Варіант 6: «Рейтинг книг»**

**Виконав:**
студент групи ФЕП-23
Корж Олексій

**Перевірив:**
проф. Свелеба С. А.

Львів – 2026

## 1. Мета роботи

Опанувати функції вищого порядку (map, filter, reduce), анонімні lambda-функції та функціональну композицію; побудувати конвеєр обробки списку книг без мутацій вхідних даних; виконати багатокритеріальне сортування та reduce-агрегацію статистичних показників.

## 2. Теоретичні відомості

**Функції вищого порядку (HOF):** функції, що приймають інші функції як аргументи або повертають їх як результат. Основними прикладами є map, filter та reduce.

**map(fn, iterable):** застосовує перетворення до кожного елемента послідовності, повертаючи новий ітератор без зміни вихідної колекції.

**filter(predicate, iterable):** відбирає лише ті елементи, для яких предикат повертає True.

**reduce(fn, iterable, [initial]):** згортає послідовність у єдине значення, послідовно застосовуючи бінарну операцію акумуляції.

**Функціональна композиція:** об’єднання функцій f(g(x)), реалізоване через допоміжні утиліти compose або лінійний конвеєр pipe.

## 3. Постановка завдання (Варіант 6)

**Тема варіанта 6:** Рейтинг книг.

**Структура запису:** Книги: id, title, genre, ratings (список оцінок користувачів).

**Конвеєр перетворень:** Обчислити середній бал, підсилити вибраний жанр на коефіцієнт (обмежений максимумом 5.0), відфільтрувати низький рейтинг, відсортувати за двома ключами (бал спадання, назва зростання) та сформувати Top-N.

**reduce-агрегація:** count (кількість), rating_sum (сума), avg_rating (середнє), min_rating, max_rating.

# 4. Хід роботи та лістинг програми

### Чисте ядро та конвеєр (core.py)

```
from __future__ import annotations

from collections.abc import Callable, Iterable
from functools import reduce
from typing import Any, TypedDict, TypeVar

A = TypeVar("A")
B = TypeVar("B")
C = TypeVar("C")


class BookRecord(TypedDict, total=False):
    id: int
    title: str
    genre: str
    ratings: list[float]
    avg_rating: float
    score: float


class Aggregation(TypedDict):
    count: int
    rating_sum: float
    avg_rating: float
    min_rating: float
    max_rating: float


def compute_avg_rating(book: BookRecord) -> BookRecord:
    ratings = book.get("ratings", [])
    avg = round(sum(ratings) / len(ratings), 3) if ratings else 0.0
    return {**book, "avg_rating": avg, "score": avg}


def boost_genre(book: BookRecord, target_genre: str, factor: float = 1.1) -> BookRecord:
    current_score = book.get("score", book.get("avg_rating", 0.0))
    if book.get("genre", "").lower() == target_genre.lower():
        boosted = round(min(5.0, current_score * factor), 3)
        return {**book, "score": boosted}
    return {**book, "score": current_score}


def filter_min_rating(
    books: Iterable[BookRecord], min_rating: float
) -> list[BookRecord]:
    return list(
        filter(
            lambda b: b.get("score", b.get("avg_rating", 0.0)) >= min_rating,
            books,
        )
    )
```

```
def sort_top_n(books: Iterable[BookRecord], n: int) -> list[BookRecord]:
    sorted_books = sorted(
        books,
        key=lambda b: (-b.get("score", 0.0), b.get("title", "")),
    )
    return sorted_books[:n]


def reduce_stats(books: Iterable[BookRecord]) -> Aggregation:
    book_list = list(books)
    if not book_list:
        return {
            "count": 0,
            "rating_sum": 0.0,
            "avg_rating": 0.0,
            "min_rating": 0.0,
            "max_rating": 0.0,
        }

    scores = [b.get("score", b.get("avg_rating", 0.0)) for b in book_list]

    initial: tuple[int, float, float, float] = (
        0,
        0.0,
        float("inf"),
        float("-inf"),
    )

    def accumulator(
        acc: tuple[int, float, float, float], score: float
    ) -> tuple[int, float, float, float]:
        cnt, r_sum, min_r, max_r = acc
        return (
            cnt + 1,
            round(r_sum + score, 3),
            min(min_r, score),
            max(max_r, score),
        )

    count, total_sum, min_val, max_val = reduce(accumulator, scores, initial)
    avg = round(total_sum / count, 3) if count > 0 else 0.0

    return {
        "count": count,
        "rating_sum": total_sum,
        "avg_rating": avg,
        "min_rating": min_val,
        "max_rating": max_val,
    }


def compose(
    f: Callable[[B], C],
    g: Callable[[A], B],
) -> Callable[[A], C]:
    return lambda x: f(g(x))


def pipe(value: A, *functions: Callable[[Any], Any]) -> Any:
    result = value
```

```
    for fn in functions:
        result = fn(result)
    return result


def process_books_pipeline(
    books: Iterable[BookRecord],
    *,
    boost_target_genre: str = "Fantasy",
    boost_factor: float = 1.1,
    min_rating: float = 4.0,
    top_n: int = 5,
) -> tuple[list[BookRecord], Aggregation]:
    step1 = list(map(compute_avg_rating, books))
    step2 = list(map(lambda b: boost_genre(b, boost_target_genre, boost_factor), step1))
    step3 = filter_min_rating(step2, min_rating)
    top_books = sort_top_n(step3, top_n)
    stats = reduce_stats(top_books)
    return top_books, stats
```

## CLI та взаємодія з даними (app.py)

```
from __future__ import annotations

import argparse
import json
from pathlib import Path

from core import Aggregation, BookRecord, process_books_pipeline


def load_books(path: Path) -> list[BookRecord]:
    with path.open(encoding="utf-8") as f:
        data = json.load(f)
    return list(data)


def render_results(books: list[BookRecord], stats: Aggregation) -> str:
    lines = [
        "=" * 65,
        "              РЕЙТИНГ КНИГ — ВАРІАНТ 6",
        "=" * 65,
        f"{'ID':<4} {'Назва книги':<35} {'Жанр':<12} {'Бал':<6}",
        "-" * 65,
    ]
    for b in books:
        title = b["title"][:33]
        score = b.get("score", 0.0)
        lines.append(f"{b['id']:<4} {title:<35} {b['genre']:<12} {score:<6.2f}")
    lines.append("-" * 65)
    lines.append("СТАТИСТИКА (REDUCE):")
    lines.append(f"  Кількість у вибірці: {stats['count']}")
    lines.append(f"  Сума балів:          {stats['rating_sum']:.2f}")
    lines.append(f"  Середній рейтинг:    {stats['avg_rating']:.2f}")
    lines.append(f"  Мінімальний бал:     {stats['min_rating']:.2f}")
    lines.append(f"  Максимальний бал:    {stats['max_rating']:.2f}")
    lines.append("=" * 65)
    return "\n".join(lines)
```

```
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--file", type=Path, default=Path("sample_data.json"))
    parser.add_argument("--top", type=int, default=5)
    parser.add_argument("--boost-genre", type=str, default="Fantasy")
    parser.add_argument("--factor", type=float, default=1.1)
    parser.add_argument("--min-rating", type=float, default=4.0)

    args = parser.parse_args()

    books = load_books(args.file)
    top_books, stats = process_books_pipeline(
        books,
        boost_target_genre=args.boost_genre,
        boost_factor=args.factor,
        min_rating=args.min_rating,
        top_n=args.top,
    )
    print(render_results(top_books, stats))


if __name__ == "__main__":
    main()
```

## Тести ядра та конвеєра (tests/test_core.py)

```
import copy

from core import (
    BookRecord,
    boost_genre,
    compose,
    compute_avg_rating,
    pipe,
    process_books_pipeline,
    reduce_stats,
    sort_top_n,
)


def test_compute_avg_rating() -> None:
    book: BookRecord = {
        "id": 1,
        "title": "Test Book",
        "genre": "Fantasy",
        "ratings": [4.0, 5.0, 4.5],
    }
    updated = compute_avg_rating(book)
    assert updated["avg_rating"] == 4.5
    assert updated["score"] == 4.5
    assert "avg_rating" not in book


def test_boost_genre() -> None:
    book1: BookRecord = {
        "id": 1,
        "title": "Fantasy Book",
        "genre": "Fantasy",
```

```
        "score": 4.0,
    }
    boosted = boost_genre(book1, "Fantasy", factor=1.2)
    assert boosted["score"] == 4.8

    book2: BookRecord = {
        "id": 2,
        "title": "Tech Book",
        "genre": "Tech",
        "score": 4.0,
    }
    not_boosted = boost_genre(book2, "Fantasy", factor=1.2)
    assert not_boosted["score"] == 4.0


def test_immutability(sample_books: list[BookRecord]) -> None:
    backup = copy.deepcopy(sample_books)
    _ = process_books_pipeline(sample_books)
    assert sample_books == backup


def test_sort_top_n() -> None:
    books: list[BookRecord] = [
        {"id": 1, "title": "B Book", "score": 4.5},
        {"id": 2, "title": "A Book", "score": 4.5},
        {"id": 3, "title": "C Book", "score": 4.9},
    ]
    top2 = sort_top_n(books, 2)
    assert len(top2) == 2
    assert top2[0]["id"] == 3
    assert top2[1]["id"] == 2


def test_reduce_stats(sample_books: list[BookRecord]) -> None:
    processed = [compute_avg_rating(b) for b in sample_books]
    stats = reduce_stats(processed)
    assert stats["count"] == 5
    assert stats["rating_sum"] > 0
    assert stats["min_rating"] <= stats["avg_rating"] <= stats["max_rating"]


def test_reduce_stats_empty() -> None:
    stats = reduce_stats([])
    assert stats["count"] == 0
    assert stats["avg_rating"] == 0.0


def test_compose_and_pipe() -> None:
    def add_ten(x: float) -> float:
        return x + 10.0

    def multiply_two(x: float) -> float:
        return x * 2.0

    composed = compose(multiply_two, add_ten)
    assert composed(5.0) == 30.0

    assert pipe(5.0, add_ten, multiply_two) == 30.0


def test_pipeline_integration(sample_books: list[BookRecord]) -> None:
```

```
    top_books, stats = process_books_pipeline(
        sample_books,
        boost_target_genre="Fantasy",
        boost_factor=1.1,
        min_rating=4.0,
        top_n=3,
    )
    assert len(top_books) <= 3
    assert stats["count"] == len(top_books)
    for b in top_books:
        assert b.get("score", 0.0) >= 4.0
```

## Property-based тести (Hypothesis) (tests/test_properties.py)

```
import copy
from collections.abc import Sequence

from core import BookRecord, process_books_pipeline
from hypothesis import given
from hypothesis import strategies as st

book_strategy = st.fixed_dictionaries(
    {
        "id": st.integers(min_value=1, max_value=10000),
        "title": st.text(min_size=1, max_size=20),
        "genre": st.sampled_from(["Fantasy", "Sci-Fi", "Tech", "Classics"]),
        "ratings": st.lists(
            st.floats(
                min_value=1.0,
                max_value=5.0,
                allow_nan=False,
                allow_infinity=False,
            ),
            min_size=1,
            max_size=10,
        ),
    }
)


@given(st.lists(book_strategy, min_size=0, max_size=20))
def test_property_no_mutation(books: Sequence[BookRecord]) -> None:
    snapshot = copy.deepcopy(list(books))
    _ = process_books_pipeline(books)
    assert books == snapshot


@given(st.lists(book_strategy, min_size=0, max_size=20))
def test_property_invariants(books: Sequence[BookRecord]) -> None:
    top_books, stats = process_books_pipeline(books, top_n=5, min_rating=3.0)
    assert len(top_books) <= 5
    assert stats["count"] == len(top_books)
    if stats["count"] > 0:
        assert stats["min_rating"] <= stats["avg_rating"] <= stats["max_rating"]
        assert stats["rating_sum"] >= 0.0
```

## 5. Результати виконання та тестування

### Запуск CLI конвеєра app.py

```
$ python app.py --top 5 --boost-genre Fantasy --factor 1.1 --min-rating 4.0
================================================================
              РЕЙТИНГ КНИГ — ВАРІАНТ 6
================================================================
ID   Назва книги                          Жанр         Бал
----------------------------------------------------------------
10   Harry Potter and the Sorcerer's S   Fantasy      5.00
6    The Fellowship of the Ring          Fantasy      5.00
1    The Hobbit                          Fantasy      5.00
7    Structure and Interpretation of C   Tech         4.90
4    1984                                Classics     4.83
----------------------------------------------------------------
СТАТИСТИКА (REDUCE):
  Кількість у вибірці: 5
  Сума балів:          24.73
  Середній рейтинг:    4.95
  Мінімальний бал:     4.83
  Максимальний бал:    5.00
================================================================
```

### Результати перевірки pytest

```
$ pytest -q
..........                                                              [100%]
10 passed in 3.88s
```

### Статична перевірка типів mypy --strict

```
$ mypy --strict .
Success: no issues found in 5 source files
```

### Перевірка форматування ruff та black

```
$ ruff check .
All checks passed!

$ black --check .
All done! ✨ 🍰 ✨
5 files would be left unchanged.
```

## 6. Висновки

У лабораторній роботі № 2 побудовано функціональний конвеєр обробки даних книг без мутації записів. Успішно використано функції вищого порядку map, filter та

reduce, продемонстровано застосування lambda та утиліт compose/pipe. Забезпечено повну стабільність тестів і строгу статичну типізацію.